Operating System Fundamentals • Chapter 06

Deadlocks

When processes wait for each other forever: what causes it, how to spot it, how to prevent it, and how to recover.

Chapter 6 of the notebook

BIG QUESTION

Two cars meet on a one-lane bridge. Neither will reverse. Now what?

Today's route

What we'll cover today

1. **The deadlock problem**
   Definition and examples
2. **Characterization**
   Four conditions and resource-allocation graphs
3. **Handling deadlocks**
   Three strategies
4. **Prevention**
   Break one condition
5. **Recovery**
   Terminate or preempt

Part 01

The deadlock problem

THINK FIRST

Four cars reach a junction at the same moment, and each waits for the car on its right. Who moves first?

discuss for one minute before the next slide

Definition

A deadlock is a set of blocked processes, each holding a resource and waiting for one held by another in the set.

Nobody can move, so nobody ever releases anything.

Two classic examples

Everyone holds one thing, everyone wants the other



Tape drives

The system has 2 tape drives. P1 and P2 each hold one, and each needs the other one too.



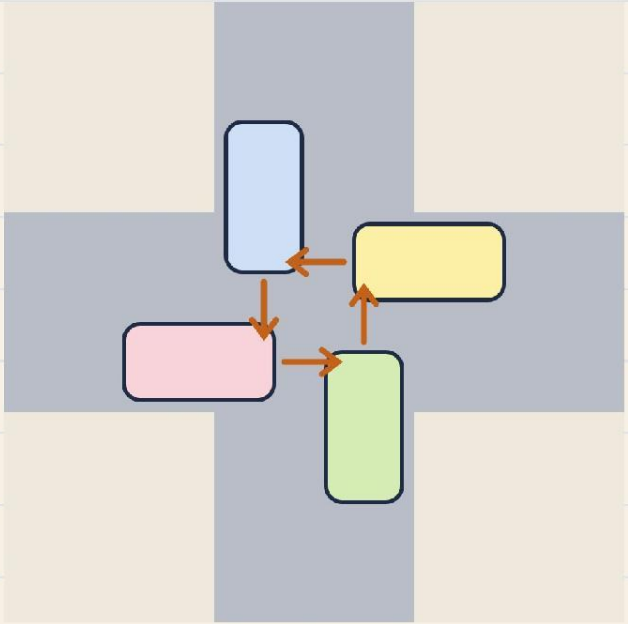
Printer and scanner

P1 holds the printer and waits for the scanner. P2 holds the scanner and waits for the printer.

both wait forever: nobody lets go first

Is this a deadlock?

Gridlock at the crossroads



Look closely

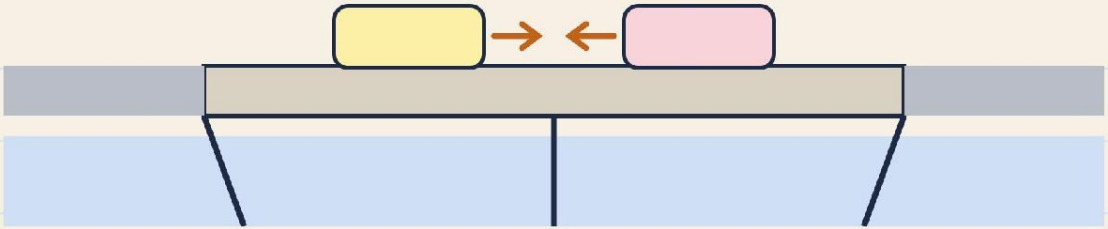
Each car occupies one quarter of the junction and needs the quarter in front of it, which another car occupies.

Your turn

Yes, it is a deadlock. How could the city prevent it? Keep your answer; we'll check it in Part 4.

Bridge crossing

A one-lane bridge: each half is a resource



Traffic in one direction only

Deadlock fix: one car backs up (preempt + roll back)

Several cars may have to back up

Starvation is possible

Part 02

Deadlock characterization

THINK FIRST

What exactly has to be true, all at once, for a deadlock to happen?

discuss for one minute before the next slide

Deadlock characterization

Four conditions, all at the same time

1 • Mutual exclusion

Only one process at a time can use a resource.

bridge: one car per lane section

2 • Hold and wait

A process holds at least one resource while waiting for more held by others.

bridge: on half, waiting for the other half

3 • No preemption

A resource is released only voluntarily, after the holder finishes its task.

bridge: nobody tows a car away

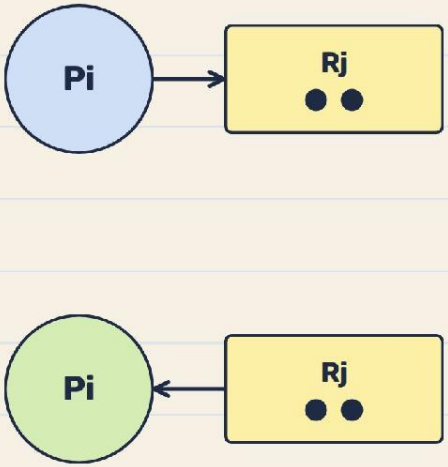
4 • Circular wait

P0 waits for P1, P1 waits for P2, …, Pn waits for P0.

bridge: each waits for the other

Resource-allocation graph

Draw it to see it



Process

A circle.

Resource type

A rectangle; one dot per instance.

Request edge Pi → Rj

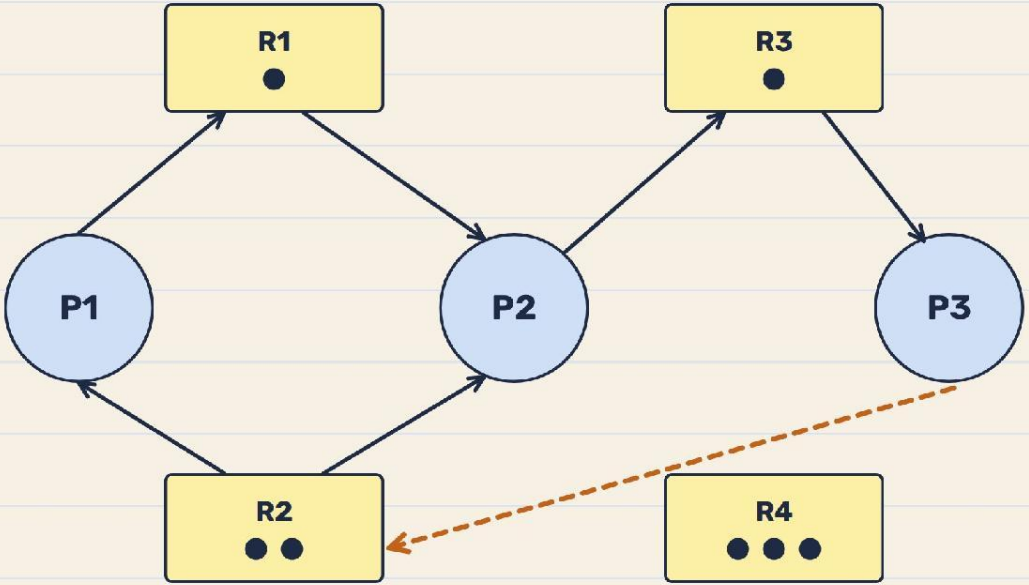
Pi is waiting for Rj. (top pair)

Assignment edge Rj → Pi

Pi is holding Rj. (bottom pair)

Resource-allocation graph

One extra edge turns it into a deadlock



Solid edges only

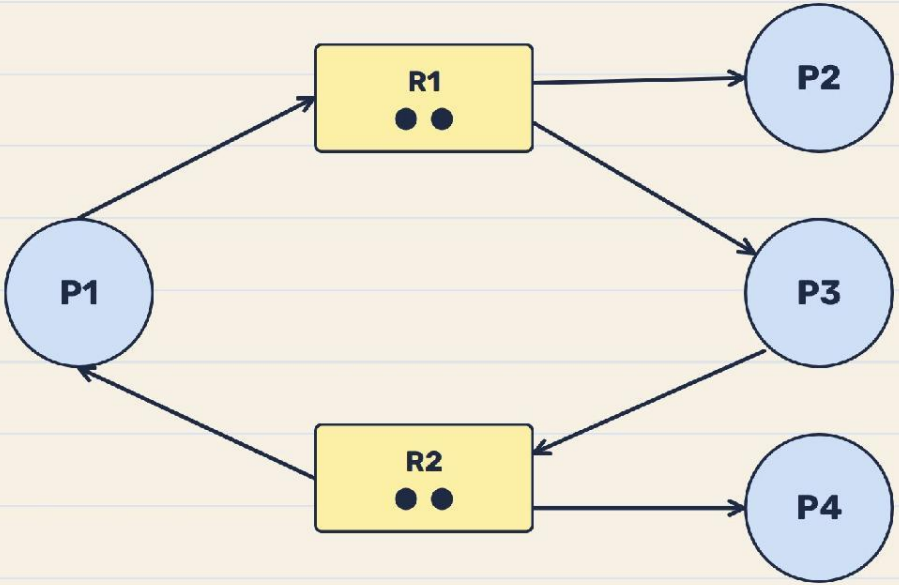
No cycle. P3 can finish, release R3, then P2 can finish, then P1. No deadlock.

Add the dashed edge

P3 now requests R2. Two cycles appear: P1 → R1 → P2 → R3 → P3 → R2 → P1 and P2 → R3 → P3 → R2 → P2. Deadlock!

Resource-allocation graph

A cycle, but no deadlock



The cycle

P1 → R1 → P3 → R2 → P1

Why it's fine

P2 and P4 are not waiting for anything. When P4 releases its R2 instance, P3 gets it, finishes, and the cycle breaks.

Basic facts

Read the graph in three steps

No cycle

No deadlock. Guaranteed.

Cycle + one instance per resource type

Deadlock. Guaranteed.

Cycle + several instances per type

Possible deadlock. Look closer.

Pop quiz • click to reveal

A resource-allocation graph has a cycle, and every resource type has exactly one instance. Is there a deadlock?

- **A** Definitely yes
- **B** Definitely no
- **C** Only if there are more than 3 processes
- **D** Only on a multiprocessor

Pop quiz • click to reveal

A resource-allocation graph has a cycle, and every resource type has exactly one instance. Is there a deadlock?

- **A** Definitely yes
- **B** Definitely no
- **C** Only if there are more than 3 processes
- **D** Only on a multiprocessor

Answer A. With one instance per resource type, a cycle means deadlock.

Part 03

Handling deadlocks

THINK FIRST

Three attitudes to any risk: make it impossible, clean up afterwards, or pretend it never happens. Which one do most OSes pick?

discuss for one minute before the next slide

Three strategies

Prevent, recover, or ignore



Never enter a deadlock

Design the system so a deadlock state is impossible (prevention).



Detect and recover

Let deadlocks happen, notice them, then fix the damage.



Ignore it

Pretend deadlocks never occur. Used by most operating systems, including UNIX and Windows.

the "ostrich algorithm": head in the sand, because real deadlocks are rare and prevention is costly

Part 04

Deadlock prevention

THINK FIRST

Break any ONE of the four conditions and deadlock becomes impossible. Which one is easiest to break?

discuss for one minute before the next slide

Deadlock prevention

Attack one condition

Condition	How to break it	Cost
Mutual exclusion	Not needed for sharable resources (e.g. read-only files)	Must still hold for non-sharable ones: usually can't break it
Hold and wait	Request everything before starting, or only request when holding nothing	Low resource utilisation; starvation possible
No preemption	If a request can't be granted, release everything held; restart when all can be regained	Only works for resources whose state is easy to save
Circular wait	Number all resource types; request only in increasing order	Programmers must follow the order

Breaking circular wait

Give every resource a number

Resource	Number
Tape drive	1
Scanner	5
Printer	12

Rule: always ask in increasing order. Holding the scanner (5), you may ask for the printer (12), never the tape drive (1).



Our printer-scanner example, fixed

Both processes must take the scanner (5) before the printer (12). Whoever gets the scanner first finishes; the other simply waits its turn.

everyone climbs in the same direction → no circle can form

Pop quiz • click to reveal

Making every process take the scanner before the printer prevents deadlock by breaking…

A Mutual exclusion

B Hold and wait

C Circular wait

D No preemption

Pop quiz · click to reveal

Making every process take the scanner before the printer prevents deadlock by breaking…

A Mutual exclusion

B Hold and wait

C Circular wait

D No preemption

Answer C. Acquiring resources in one fixed global order makes a circular wait impossible.

Part 05

Recovery from deadlock

THINK FIRST

The deadlock has happened. Someone has to be sacrificed. How do you choose who?

discuss for one minute before the next slide

Recovery option 1

Process termination



Abort them all

Kill every deadlocked process. Simple, but expensive: all their work is lost.



One at a time

Abort one process, check again, repeat until the cycle is gone.

Who goes first? Consider:

- Priority
- Time computed and time left
- Resources used
- Resources still needed
- How many must be terminated
- Interactive or batch?

Recovery option 2

Resource preemption



Select a victim

Choose whose resources to take, minimising cost.



Roll back

Return the victim to some safe state and restart it from there.



Avoid starvation

The same process may always be picked. Include the number of rollbacks in the cost.

↺ Recall · starvation & aging, Ch 5

Same fix, different setting: the longer you've suffered, the less likely you're picked again.

Recap

The bridge, condition by condition

Condition	On the bridge	Prevention idea
Mutual exclusion	One car per lane section	(can't share a lane)
Hold and wait	A car sits on one half and waits for the other	Enter only if the whole bridge is free
No preemption	Nobody can remove a car	Allow a tow truck (roll back)
Circular wait	Each car waits for the other	Traffic light: one direction at a time

Wrap-up

What to remember

OS JOURNAL · CHAPTER 6

- ✓ Deadlock: each process holds a resource and waits for another's.
- ✓ No cycle in the graph → no deadlock; cycle + single instances → deadlock.
- ✓ Ordering resources breaks circular wait: the most practical prevention.
- ✓ Four conditions must all hold: mutual exclusion, hold and wait, no preemption, circular wait.
- ✓ Strategies: prevent, detect and recover, or ignore (most OSes).
- ✓ Recovery: terminate processes, or preempt resources and roll back.

Next up *Chapter 7 · Memory Management: sharing memory without chaos*